

Grupa P2
Skład grupy:
Oleksandr Barabash – Koordynator
Yurii Morskyi
Bohdan Krasovskyi
Stanislav Smaha
Bohdan Kliukovskyi

Taskora

User Requirements Document

Data ostatniej modyfikacji: 12.04.2026
Wersja: finalna

Spis treści

1 Wprowadzenie.....	2
1.1 Cel dokumentu.....	2
1.2 Zakres oprogramowania.....	2
1.3 Definicje i skróty.....	2
1.3.1 Definicje.....	2
1.3.2 Skróty.....	5
1.4 Odnosiniki.....	6
2 Opis ogólny.....	7
2.1 Perspektywa systemu.....	7
2.2 Ogólne możliwości systemu.....	7
2.3 Ogólne ograniczenia.....	8
2.4 Charakterystyka użytkownika.....	8
2.5 Środowisko pracy systemu i jego architektura.....	9
2.6 Założenia i zależności.....	9
3 Wymagania szczegółowe.....	10
3.1 Wymagania funkcjonalne.....	10
3.1.1 Obszar Zarządzania Użytkownikami i Uprawnieniami.....	10
3.1.2 Obszar Zarządzania Projektami i Roadmapami.....	10
3.1.3 Obszar Zarządzania Zadaniem i Cyklem Agile.....	11
3.1.4 Obszar Tablic Interaktywnych i Raportowania.....	12
3.1.5 Obszar Komunikacji, Powiadomień i Integracji.....	12
3.2 Wymagania niefunkcjonalne.....	14
3.2.1 Obszar Dostępności i Niezawodności.....	14
3.2.2 Obszar Bezpieczeństwa i Zgodności.....	15
3.2.3 Obszar Kompatybilności i Środowiska Klienta.....	15
3.2.4 Obszar Skalowalności i Architektury.....	16
3.2.5 Obszar Wydajności i Integracji.....	17
3.3 Wymagania zgodności.....	18
3.3.1 Obszar Zgodności Prawnej i Ochrony Danych.....	18
3.3.2 Obszar Zgodności z Politykami Bezpieczeństwa Korporacyjnego.....	18
3.3.3 Obszar Zgodności ze Standardami Webowymi i Przeglądarkami.....	19
3.3.4 Obszar Zgodności Architektonicznej i Interfejsów API.....	20
3.3.5 Obszar Zgodności z Metodami Zarządzania Projektami.....	21

1.Wprowadzenie

1.1.Cel dokumentu

Celem niniejszego dokumentu jest przedstawienie wymagań użytkownika dla systemu informatycznego **Taskora**, który został zaprojektowany jako narzędzie do zarządzania projektami oraz zadaniami w organizacji. Dokument URD określa potrzeby użytkowników, funkcjonalności systemu oraz wymagania dotyczące jego działania. System **Taskora** ma wspierać zarządzanie projektami w organizacjach, szczególnie w firmach z branży informatycznej i technologicznej. Umożliwia planowanie pracy zespołu, tworzenie projektów, zarządzanie zadaniami oraz monitorowanie postępów realizacji projektów.

Dokument jest skierowany do zespołu projektowego, analityków systemowych, programistów, menedżerów projektów oraz przyszłych użytkowników systemu. Wymagania zawarte w dokumencie będą wykorzystywane podczas projektowania, implementacji oraz testowania systemu. Dokument ma również na celu zapewnienie wspólnego zrozumienia wymagań przez wszystkich uczestników projektu oraz ograniczenie ryzyka nieporozumień podczas jego realizacji.

1.2.Zakres oprogramowania

Taskora to system informatyczny wspierający zarządzanie projektami i zadaniami w zespołach projektowych, szczególnie w środowisku IT. System umożliwia tworzenie projektów, zarządzanie zadaniami oraz monitorowanie postępów prac. **Taskora** wspiera metodyki zwinne, takie jak Scrum i Kanban, umożliwiając zarządzanie backlogiem projektu, planowanie sprintów oraz przypisywanie zadań do epików i iteracji. System oferuje wizualizację pracy zespołu za pomocą tablic Kanban oraz generowanie raportów dotyczących postępów prac, w tym raportów wydajności zespołu (np. velocity) i roadmapy projektu.

1.3.Definicje i skróty

1.3.1.Definicje

Agile - Podejście do zarządzania projektami i tworzenia oprogramowania oparte na iteracyjnym rozwoju, elastycznym planowaniu oraz ciągłej współpracy zespołu i interesariuszy.

Backlog - Uporządkowana według priorytetów lista funkcjonalności, wymagań oraz zadań planowanych do realizacji w projekcie.

Chmura - model przetwarzania danych oparty na użytkowaniu zasobów obliczeniowych (serwerów, pamięci masowej) dostarczanych przez zewnętrznego dostawcę przez Internet, co umożliwia dynamiczne skalowanie systemu.

Drag-and-drop - technika interakcji z graficznym interfejsem użytkownika, polegająca na chwyceniu obiektu (np. karty zadania na tablicy Kanban) za pomocą myszy lub dotyku, przesunięciu go w inne miejsce i upuszczeniu w celu zmiany jego statusu lub przypisania.

Epic - Duża funkcjonalność systemu obejmująca zbiór powiązanych zadań, która może być realizowana w kilku etapach projektu.

Implementacja - Etap procesu tworzenia oprogramowania polegający na realizacji zaprojektowanych funkcjonalności poprzez pisanie i integrację kodu źródłowego.

Iteracja - Powtarzalny cykl pracy w projekcie, w którym realizowany jest określony zestaw zadań prowadzących do stopniowego rozwijania produktu.

Kamień milowy - Ważny punkt kontrolny w harmonogramie projektu oznaczający zakończenie określonego etapu prac.

Kanban - Wizualne narzędzie do zarządzania pracą zespołu przedstawiające status realizacji zadań w kolejnych etapach procesu.

Oprogramowanie klienckie - komponent systemu informatycznego uruchamiany na urządzeniu użytkownika końcowego, który komunikuje się z serwerem w celu uzyskania dostępu do zasobów i funkcjonalności.

Repozytorium wiedzy - centralna baza danych lub system przechowywania informacji, w którym gromadzone, organizowane i udostępniane są dokumenty, dane projektowe oraz historia zmian w zadaniach.

Roadmapa - Plan rozwoju projektu przedstawiający główne cele, etapy oraz kierunek rozwoju systemu w określonym czasie.

Scrum - Ramy postępowania (framework) w zwinnym zarządzaniu projektami polegające na iteracyjnej realizacji pracy w krótkich cyklach zwanych sprintami oraz na stałej współpracy zespołu projektowego.

Skalowalność - zdolność systemu do efektywnej obsługi rosnącej liczby użytkowników, projektów i danych poprzez zwiększanie zasobów infrastruktury bez utraty wydajności.

Software - Zbiór programów komputerowych, danych oraz powiązanej dokumentacji umożliwiających wykonywanie określonych funkcji przez system komputerowy.

Sprint - Krótki, określony czasowo etap pracy w ramach Scrum, w którym zespół realizuje zaplanowany zestaw zadań, tworząc gotowy przyrost produktu.

Velocity - Miara wydajności zespołu projektowego w metodykach zwinnych, określająca ilość pracy (np. liczby zadań lub punktów historyjek) ukończonych w jednym sprincie.

Wirtualizacja - technologia umożliwiająca uruchomienie wielu systemów operacyjnych lub aplikacji na jednym fizycznym serwerze, co pozwala na optymalne wykorzystanie zasobów sprzętowych infrastruktury.

Bug - Obiekt roboczy w systemie zarządzania projektami reprezentujący wykryty błąd lub defekt w oprogramowaniu, wymagający naprawy przez zespół projektowy.

Core Web Vitals - Zestaw kluczowych metryk wydajności aplikacji webowej, zdefiniowanych przez Google, mierzących szybkość ładowania, stabilność wizualną oraz responsywność interfejsu na interakcje użytkownika.

Deep link - Bezpośredni adres URL wskazujący na konkretny widok lub zasób wewnątrz aplikacji webowej, umożliwiający udostępnianie linków do poszczególnych stron, tablic czy zadań.

Fallback - Mechanizm awaryjny w aplikacji, polegający na zastosowaniu alternatywnego rozwiązania technicznego w przypadku, gdy docelowa funkcjonalność nie jest wspierana przez środowisko użytkownika.

Feature detection - Technika programistyczna polegająca na sprawdzaniu w czasie działania aplikacji, czy przeglądarka użytkownika obsługuje daną funkcjonalność, zamiast polegania na identyfikacji wersji przeglądarki.

Responsywność - Właściwość interfejsu użytkownika polegająca na automatycznym dostosowywaniu układu i elementów graficznych do rozdzielczości oraz rozmiaru ekranu urządzenia.

Story Points - Jednostka estymacji złożoności zadania stosowana w metodykach zwinnych, służąca do szacowania nakładu pracy niezbędnego do realizacji danego elementu backlogu.

User Story - Element backlogu opisujący funkcjonalność systemu z perspektywy użytkownika końcowego, definiujący kto, co i po co chce osiągnąć.

1.3.2.Skróty

URD - User Requirements Document

JSON - JavaScript Object Notation

SMTP - Simple Mail Transfer Protocol

SPOF - Single Point of Failure

API - Application Programming Interface

CI/CD - Continuous Integration / Continuous Deployment

SPA - Single Page Application

HTML - HyperText Markup Language

HTTP – HyperText Transfer Protocol

HTTPS - HyperText Transfer Protocol Secure

RDBMS - Relational Database Management System

IT - Information Technology

REST - Representational State Transfer

RBAC - Role-Based Access Control

PM - Project Manager

PoLP - Principle of Least Privilege

CLS - Cumulative Layout Shift

CSP - Content Security Policy

CSS - Cascading Style Sheets

INP - Interaction to Next Paint

LCP - Largest Contentful Paint

MDN -Mozilla Developer Network

UTF-8 - Unicode Transformation Format 8-bit

W3C - World Wide Web Consortium

WAI-ARIA - Web Accessibility Initiative - Accessible Rich Internet Applications

WCAG - Web Content Accessibility Guidelines

WHATWG - Web Hypertext Application Technology Working Group

IdP – Identity Provider

SSO – Single Sign-On

MFA – Multi-Factor Authentication

JWT – JSON Web Token

CORS – Cross-Origin Resource Sharing

SQL – Structured Query Language
XSS – Cross-Site Scripting
OWASP – Open Web Application Security Project
Audit Trail – dziennik audytowy
WAF – Web Application Firewall
DDoS – Distributed Denial of Service
VPC – Virtual Private Cloud
AES-256 – Advanced Encryption Standard 256-bit
KMS – Key Management Service
TLS – Transport Layer Security
mTLS – Mutual Transport Layer Security
SIEM – Security Information and Event Management
CVE – Common Vulnerabilities and Exposures

1.4.Odnośniki

- [1] **The Scrum Guide** – Ken Schwaber & Jeff Sutherland. Oficjalny przewodnik po metodyce Scrum. Dostępne na: <https://scrumguides.org/>
- [2] **Rozporządzenie o Ochronie Danych Osobowych (RODO)** – Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. (wytyczne dotyczące przetwarzania danych osobowych użytkowników). Dostępne na: <https://eur-lex.europa.eu/legal-content/PL/TXT/HTML/?uri=CELEX:32016R0679>
- [3] **Specyfikacja protokołu SMTP (RFC 5321)** – Internet Engineering Task Force (IETF). Dokumentacja techniczna dla niezawodnej komunikacji e-mail. Dostępne na: <https://datatracker.ietf.org/doc/html/rfc5321>
- [4] **Standardy W3C (HTML5)** – World Wide Web Consortium. Wytyczne dotyczące budowy nowoczesnych aplikacji webowych (SPA). Dostępne na: <https://html.spec.whatwg.org/>

2.Opis ogólny

2.1.Perspektywa systemu

System **Taskora** jest autonomicznym rozwiązaniem informatycznym klasy Project Management Software, zaprojektowanym do kompleksowego wspomagania zarządzania cyklem życia projektów programistycznych. System nie wymaga integracji z nadrzędnymi systemami korporacyjnymi do poprawnego działania, jednak udostępnia interfejsy umożliwiające wymianę danych z otoczeniem:

- **Interfejsy zewnętrzne:** System udostępnia publiczny, dokumentowany interfejs API oparty na architekturze REST oraz formacie wymiany danych JSON. Umożliwia to dwustronną komunikację z systemami kontroli wersji (np. GitHub, GitLab) oraz platformami CI/CD w celu automatycznej aktualizacji statusów zadań.
- **Komunikacja sieciowa:** System integruje się z zewnętrznymi serwerami pocztowymi (protokół SMTP), co pozwala na automatyczne generowanie i wysyłkę powiadomień o kluczowych zdarzeniach w projektach (np. przypisanie zadania, zbliżający się termin).
- **Zarządzanie danymi:** **Taskora** pełni rolę centralnego repozytorium wiedzy o postępach prac. System agreguje dane wejściowe od użytkowników i przetwarza je w spójne raporty, przechowując informacje w relacyjnej strukturze bazy danych, co zapewnia integralność i trwałość informacji projektowych.

2.2.Ogólne możliwości systemu

- **Zarządzanie strukturą i uprawnieniami:** tworzenie wielopoziomowej hierarchii projektów, definiowanie ról użytkowników (Administrator, PM, Developer) oraz granularne zarządzanie dostępem do danych.
- **Obsługa cyklu życia zadań:** pełna ewidencja i edycja zadań, błędów oraz epików (epics) wraz z przypisywaniem priorytetów i terminów.
- **Wizualizacja procesów Agile:** interaktywne tablice Kanban i Scrum z funkcjonalnością "przeciągnij i upuść" (drag-and-drop) do monitorowania statusów prac w czasie rzeczywistym.
- **Planowanie długoterminowe (Roadmap):** generowanie wizualnych map drogowych projektu, umożliwiających śledzenie kamieni milowych i zależności czasowych.
- **Komunikacja i powiadomienia:** system komentarzy wewnątrz zadań oraz automatyczne powiadomienia e-mail o zmianach statusów i przypisaniach.

2.3.Ogólne ograniczenia

Rozwój i działanie systemu podlegają następującym ograniczeniom biznesowym i technicznym:

Architektura webowa i przenośność (Portability): System musi funkcjonować jako aplikacja przeglądarkowa kompatybilna z najnowszymi wersjami silników Chromium, Gecko i WebKit. Aby zapewnić natychmiastowy dostęp dla rozproszonych zespołów pracujących na różnych systemach operacyjnych (Windows, macOS, Linux) bez konieczności instalacji, wdrażania i aktualizacji oprogramowania klienckiego.

Bezpieczeństwo i zgodność (Security & Compliance): Cała komunikacja sieciowa musi być szyfrowana, a hasła i wrażliwe dane w bazie odpowiednio zabezpieczone. System musi wspierać integrację z korporacyjnymi dostawcami tożsamości. Aplikacja będzie przetwarzać wrażliwe dane projektowe, tajemnice handlowe oraz dane osobowe pracowników, co wymusza zgodność z rygorystycznymi politykami bezpieczeństwa firm oraz regulacjami prawnymi.

Dostępność (Availability): System powinien zapewniać wysoką dostępność rdzenia usług na poziomie co najmniej 99,9% w skali miesiąca, z wyłączeniem przerw spowodowanych awariami usług zewnętrznych (np. zewnętrznych serwerów SMTP). W celu zwiększenia niezawodności systemu należy stosować mechanizmy takie jak kopie zapasowe bazy danych, monitorowanie działania serwera oraz możliwość skalowania infrastruktury w przypadku wzrostu liczby użytkowników.

Standaryzacja integracji (Interfejsy): Oprogramowanie musi bezwzględnie udostępniać publiczne, wersjonowane API (RESTful). Wynika to z konieczności bezproblemowej wymiany danych z zewnętrznymi, już istniejącymi w firmach narzędziami (np. systemy CI/CD, komunikatory typu Slack/MS Teams, systemy kontroli wersji takie jak GitLab/Bitbucket)

2.4.Charakterystyka użytkowników

- **Administrator:** Zarządza konfiguracją systemu oraz użytkownikami, w tym tworzeniem kont i przypisywaniem ról.
- **Project Manager / Scrum Master:** Zarządza projektami, planuje pracę zespołu, przypisuje zadania oraz monitoruje postęp realizacji projektu.
- **Developer / Wykonawca:** Realizuje przypisane zadania, aktualizuje ich status oraz dodaje komentarze i informacje o postępie prac.
- **Klient / Obserwator:** Posiada dostęp tylko do wybranych projektów w trybie odczytu, aby monitorować postęp prac.

2.5. Środowisko pracy systemu i jego architektura

System opiera się na nowoczesnej architekturze trójwarstwowej, co zapewnia skalowalność i bezpieczeństwo danych:

- **Warstwa Klienta (Frontend):** Aplikacja typu SPA uruchamiana po stronie użytkownika. Wymaga nowoczesnej przeglądarki internetowej wspierającej standardy HTML5 i JavaScript.
- **Warstwa Logiki (Backend):** Serwer aplikacji odpowiedzialny za autoryzację, walidację danych oraz przetwarzanie reguł biznesowych zarządzania projektami. Serwer komunikuje się z klientem za pomocą szyfrowanego protokołu HTTPS.
- **Warstwa Danych:** System zarządzania relacyjną bazą danych (RDBMS), która odpowiada za trwałość i spójność informacji o projektach, sprintach i użytkownikach.

Środowisko operacyjne po stronie serwera opiera się na rozwiązaniach chmurowych lub wirtualizacji, co umożliwia dynamiczne skalowanie zasobów w zależności od obciążenia.

2.6. Założenia i zależności

Poprawne funkcjonowanie i wdrożenie systemu **Taskora** opiera się na następujących założeniach:

- **Infrastruktura sieciowa:** Użytkownik końcowy musi posiadać stabilne połączenie z siecią Internet. Minimalna przepustowość powinna pozwalać na płynną obsługę interaktywnych tablic Kanban (Drag-and-Drop).
- **Usługi zewnętrzne:** Niezawodność powiadomień e-mail jest bezpośrednio zależna od dostępności i poprawnej konfiguracji zewnętrznego serwera pocztowego.
- **Zgodność przeglądarek:** Zakłada się, że użytkownicy korzystają z aktualnych wersji przeglądarek (Chrome, Firefox, Safari, Edge), co jest niezbędne do poprawnego renderowania roadmpy i wykresów.
- **Kompetencje użytkowników:** Przyjmuje się, że osoby korzystające z systemu posiadają elementarną wiedzę na temat zarządzania zadaniami oraz obsługi interfejsów webowych.

3.Wymagania szczegółowe

3.1.Wymagania funkcjonalne

3.1.1.Obszar Zarządzania Użytkownikami i Uprawnieniami (autor: Oleksandr Barabash)

1. System musi umożliwiać Administratorowi utworzenie nowego konta użytkownika poprzez podanie następujących danych: nazwa użytkownika (login), adres e-mail, imię, nazwisko, rola oraz hasło tymczasowe.
2. System musi wymuszać na użytkowniku zmianę hasła tymczasowego na własne podczas pierwszego pomyślnego logowania do systemu.
3. System musi zarządzać uprawnieniami w oparciu o model ról (RBAC). Zestawy uprawnień zdefiniowane dla poszczególnych ról muszą być rozłączne i nie mogą się powielać.
4. System musi umożliwiać Użytkownikowi edycję podstawowych danych profilowych (imię, nazwisko) w dowolnym momencie.
5. System musi wymuszać stosowanie polityki silnych haseł. Nowe hasło musi składać się z minimum 8 znaków i zawierać co najmniej jedną wielką literę, jedną małą literę, jedną cyfrę oraz jeden znak specjalny.
6. System musi umożliwiać Użytkownikowi zmianę adresu e-mail. Po zainicjowaniu zmiany, system musi wysłać link weryfikacyjny na nowy adres e-mail, a sama zmiana następuje dopiero po kliknięciu w ten link. System musi jednocześnie wysłać powiadomienie informacyjne o próbie zmiany na dotychczasowy (stary) adres e-mail Użytkownika.
7. System musi umożliwiać zalogowanemu Użytkownikowi zmianę hasła do konta. W celu autoryzacji tej operacji, system musi wymagać poprawnego podania aktualnego hasła, a następnie dwukrotnego wprowadzenia nowego hasła.
8. System musi umożliwiać Administratorowi dezaktywację konta użytkownika (tzw. soft-delete), co uniemożliwi mu logowanie, ale zachowa spójność danych historycznych w systemie. Przed wykonaniem tej akcji system musi wymagać od Administratora dodatkowego potwierdzenia poprzez okno dialogowe.
9. System musi umożliwiać Administratorowi zmianę roli przypisanej do istniejącego konta użytkownika.
10. System musi wymuszać, aby każdy użytkownik posiadał dokładnie jedną z ról w danym momencie: Administrator, Manager, Developer, Klient.

3.1.2.Obszar Zarządzania Projektami i Roadmapami (autor: Stanislav Smaha)

1. System musi umożliwiać uprawnionym rolom (np. Project Manager) tworzenie nowych projektów oraz definiowanie ich kategorii lub typu (np. rozwój oprogramowania, aplikacje mobilne, utrzymanie systemów).
2. System musi umożliwiać definiowanie podstawowych parametrów projektu, w tym nazwy, opisu, celów biznesowych, daty rozpoczęcia oraz planowanej daty zakończenia.
3. System musi umożliwiać tworzenie wielopoziomowej hierarchii struktury projektu obejmującej poziomy: projekt, epik (epic), zadanie (task) oraz podzadanie (sub-task), z zachowaniem relacji nadrzędny–podrzędny.²

4. System musi umożliwiać tworzenie epików jako nadrzędnych elementów projektu, grupujących powiązane zadania oraz umożliwiać zarządzanie ich atrybutami (nazwa, opis, priorytet, status).
5. System musi umożliwiać definiowanie kamieni milowych (milestones) projektu wraz z przypisaniem dat oraz powiązanych epików i zadań.
6. System musi umożliwiać przypisywanie członków zespołu do projektu wraz z określeniem ich roli oraz poziomu dostępu w ramach danego projektu.
7. System musi udostępniać pulpit (dashboard) prezentujący projekty użytkownika wraz z ich statusem, liczbą aktywnych zadań oraz poziomem realizacji.
8. System musi umożliwiać wizualizację zależności czasowych pomiędzy zadaniami i epikami na roadmapie projektu (np. w formie wykresu Gantta lub osi czasu), w tym relacji typu „zależne od” oraz „blokujące”.
9. System musi umożliwiać monitorowanie postępu realizacji projektu poprzez prezentację wskaźników: procent ukończenia projektu, liczba zadań w realizacji oraz stopień realizacji epików w czasie rzeczywistym.
10. System musi umożliwiać monitorowanie postępu realizacji projektu poprzez prezentację wskaźników: procent ukończenia projektu, liczba zadań w realizacji oraz stopień realizacji epików w czasie rzeczywistym.

3.1.3.Obszar Zarządzania Zadaniem i Cyklem Agile (autor: Yurii Morskyi)

1. System musi umożliwiać użytkownikom tworzenie nowych obiektów typu „Zadanie” (Task) oraz „Błąd” (Bug). Podczas tworzenia system musi wymagać wypełnienia pól: Tytuł oraz Typ, a także pozwalać na uzupełnienie opcjonalnego Opisu. Użytkownik musi mieć możliwość edycji tych pól po utworzeniu zgłoszenia.
2. System musi umożliwiać przypisanie do każdego Zadania, Błędu oraz User Story wartości estymacji wyrażonej w punktach (Story Points). Wartość estymacji musi być edytowalna przez Developera oraz Scrum Mastera.
3. System musi definiować następujące statusy dla obiektów roboczych: „Nowy”, „W trakcie”, „W przeglądzie”, „Zrobione”. Przejścia między statusami muszą być ograniczone do zdefiniowanego przepływu (np. „Nowy” → „W trakcie” → „W przeglądzie” → „Zrobione”). System musi blokować przejścia niezgodne z przepływem.
4. Widok Backlogu musi umożliwiać: ręczną zmianę kolejności elementów metodą drag-and-drop, filtrowanie według typu, priorytetu, przypisanego wykonawcy oraz Epiku, a także wyszukiwanie pełnotekstowe po tytule i opisie.
5. System musi umożliwiać określenie poziomu priorytetu (np. Krytyczny, Wysoki, Średni, Niski) oraz precyzyjnego terminu realizacji (Deadline) dla każdego Zadania i Błędu.
6. System musi umożliwiać Scrum Masterowi tworzenie nowych Sprintów poprzez zdefiniowanie: Nazwy Sprintu, Daty rozpoczęcia, Daty zakończenia oraz Celu Sprintu. System musi weryfikować i blokować tworzenie Sprintów, których ramy czasowe nakładają się na siebie w ramach jednego zespołu.
7. System musi umożliwiać Scrum Masterowi ręczne uruchamianie (zmiana statusu na „Aktywny”) oraz zamykanie Sprintów. Przy zamykaniu system musi wyświetlić podsumowanie ukończonych i nieukończonych elementów oraz umożliwić Scrum Masterowi wybór: przeniesienie niezakończonych zadań do następnego Sprintu albo zwrócenie ich do Backlogu.

8. System musi pozwalać Developerom na edycję następujących atrybutów przypisanych im Zadań: Opis, Priorytet, Story Points, Status oraz Termin realizacji.
9. System musi automatycznie rejestrować historię zmian dla każdego obiektu roboczego. W widoku szczegółów system musi wyświetlać: datę i czas modyfikacji, imię i nazwisko autora zmiany, modyfikowany atrybut oraz wartość pola przed i po modyfikacji.
10. System musi pozwalać na przypisanie Zadania lub Błędu do konkretnego wykonawcy (Developera) w zespole projektowym.

3.1.4.Obszar Tablic Interaktywnych i Raportowania (autor:Bohdan Kliukovskyi)

1. System musi udostępniać interaktywne tablice zwinne w dwóch trybach: Scrum (oparte na iteracjach) oraz Kanban (przepływ ciągły).
2. System musi umożliwiać konfigurację kolumn na tablicach w celu dostosowania przepływu pracy (workflow) do specyfiki projektu.
3. System musi pozwalać na pełne zarządzanie cyklem życia Sprintu, w tym jego tworzenie, planowanie, uruchamianie i zamykanie.
4. System musi umożliwiać zmianę statusu zadań przy użyciu mechanizmu drag-and-drop (przeciągnij i upuść).
5. System musi odświeżać widok tablic w czasie rzeczywistym dla wszystkich użytkowników (synchronizacja real-time).
6. System musi oferować zaawansowane filtrowanie zadań według przypisanej osoby, statusu, Epiku oraz priorytetu.
7. System musi automatycznie generować wykres spalania (Burndown Chart) prezentujący postęp prac w ramach aktywnego Sprintu.
8. System musi wyliczać i wizualizować raport wydajności zespołu (Velocity Chart) na podstawie danych historycznych.
9. System musi agregować dane o projektach i generować ogólne raporty analityczne dotyczące obciążenia zespołu.
10. System musi pozwalać na eksportowanie raportów i zestawień do plików zewnętrznych w formatach PDF, CSV oraz XLSX.

3.1.5.Obszar Komunikacji, Powiadomień i Integracji (autor: Bohdan Krasovskyi)

1. System musi udostępniać możliwość dodawanie komentarzy w czasie rzeczywistym, z obsługą formatowania tekstu (Markdown) oraz załączeniem plików graficznych.
2. Interfejs musi pozwalać rolom Developer oraz Project Manager na monitorowanie historii zmian zadania bezpośrednio w widoku szczegółowym, zapewniając transparentność postępów.
3. System musi generować automatyczne powiadomienia e-mail w formacie HTML, wysyłane niezwłocznie (w czasie < 60s) do wykonawcy po przypisaniu go do nowego zgłoszenia.
4. Mechanizm subskrypcji musi wysyłać powiadomienia do wszystkich obserwujących (watchers) za każdym razem, gdy nastąpi zmiana statusu lub priorytetu zadania.

5. System musi generować alerty przypominające o zbliżającym się terminie realizacji zadania (Deadline). Powiadomienia muszą być wysyłane na 24 godziny oraz na 2 godziny przed upływem terminu.
6. System musi wysyłać bezpieczne powiadomienia e-mail o kluczowych zmianach w projektach, aby użytkownicy byli na bieżąco z przypisanymi zadaniami.
7. System musi umożliwiać łatwe i bezpieczne łączenie się z innymi aplikacjami używanymi w firmie, aby użytkownicy mogli przysyłać dane projektowe do zewnętrznych narzędzi bez konieczności ich ręcznego kopiowania.
8. System musi zapewniać sprawną i obustronną wymianę informacji z zewnętrznym oprogramowaniem, dbając o to, by dane o zadaniach i postępach prac były zawsze aktualne we wszystkich zintegrowanych systemach.
9. System musi automatycznie aktualizować status zadania, gdy programista zakończy pracę nad kodem, co eliminuje konieczność ręcznego "odklikiwania" zadań.
10. System musi informować użytkownika o wynikach automatycznych testów i wdrożeń bezpośrednio w widoku zadania, ułatwiając monitorowanie postępów prac technicznych.

3.2.Wymagania niefunkcjonalne

3.2.1.Obszar Dostępności i Niezawodności (Availability & Reliability) (autor: Oleksandr Barabash)

1. System musi zapewniać dostępność usług własnych na poziomie minimum 99,9% w skali miesiąca kalendarzowego. Do czasu niedostępności (downtime) nie wlicza się awarii niezależnych systemów zewnętrznych oraz zaplanowanych okien serwisowych.
2. System musi automatycznie tworzyć pełną kopię zapasową (backup) bazy danych co najmniej raz na 12 godzin. Kopie zapasowe muszą być przechowywane w bezpiecznej lokalizacji przez okres minimum 30 dni. System lub procedury operacyjne muszą zapewniać możliwość przetestowania integralności i poprawnego odtworzenia danych z kopii zapasowej.
3. Infrastruktura serwerowa systemu musi być objęta ciągłym monitoringiem, który śledzi kluczowe parametry wydajnościowe (m.in. obciążenie CPU, zużycie pamięci RAM, dostępną przestrzeń dyskową). W przypadku przekroczenia krytycznych wartości lub awarii, system monitorujący musi automatycznie wysłać powiadomienia (alerty) do Administratorów.
4. System musi posiadać scentralizowany mechanizm logowania zdarzeń aplikacyjnych i błędów. Logi techniczne nie mogą zawierać danych wrażliwych (np. haseł w postaci jawnej) i muszą być przechowywane przez okres co najmniej 14 dni w celu ułatwienia diagnozy awarii.
5. W przypadku wykrycia utraty połączenia z siecią Internet po stronie użytkownika, system musi zablokować możliwość wysyłania formularzy oraz wyświetlić czytelny komunikat o braku dostępu do sieci.
6. Architektura systemu musi być pozbawiona pojedynczych punktów awarii (SPOF) na poziomie infrastruktury sprzętowej i sieciowej, poprzez zastosowanie redundancji w środowisku chmurowym lub wirtualizacyjnym.
7. System musi wykorzystywać mechanizm transakcji bazodanowych (zgodnych z ACID) dla wszystkich operacji modyfikujących dane projektowe. W przypadku błędu podczas wieloetapowego zapisu, system musi wycofać wszystkie zmiany (rollback), aby zachować pełną spójność danych.
8. System musi być w pełni funkcjonalny w trybie autonomicznym (stand-alone), co oznacza, że do realizacji swoich podstawowych procesów nie wymaga połączenia z platformami trzecimi. Jednocześnie architektura systemu musi pozwalać na opcjonalną integrację z zewnętrznymi narzędziami (np. Microsoft Teams), których ewentualna awaria lub odłączenie nie zaburzy głównego działania systemu.
9. System musi posiadać mechanizm limitowania zapytań (Rate Limiting) na poziomie API. W przypadku przekroczenia dozwolonego limitu żądań z jednego adresu IP lub konta, system musi odrzucić kolejne zapytania, chroniąc zasoby serwera przed wyczerpaniem i zachowując dostępność dla innych użytkowników.
10. Architektura systemu musi wspierać skalowanie poziome (horizontal scaling) oraz równoważenie obciążenia (load balancing). System musi automatycznie uruchomić dodatkowe węzły obliczeniowe, gdy średnie obciążenie procesora (CPU) lub zużycie pamięci (RAM) przekroczy 80% przez czas dłuższy niż 5 minut, gwarantując utrzymanie średniego czasu odpowiedzi (Response Time) na poziomie poniżej 2 sekund.

3.2.2.Obszar Bezpieczeństwa i Zgodności (Security & Compliance) (autor: Stanislav Smaha)

1. System musi umożliwiać opcjonalną integrację z korporacyjnymi dostawcami tożsamości (IdP) wykorzystując standardy branżowe (jako alternatywę dla wbudowanego systemu autoryzacji SSO).
2. System musi natywnie wspierać i umożliwiać Administratorom wymuszenie stosowania uwierzytelniania wieloskładnikowego (MFA - Multi-Factor Authentication) dla kont o podwyższonych uprawnieniach.
3. Hasła użytkowników w bazie danych muszą być przechowywane wyłącznie w postaci zhashowanej z użyciem silnych algorytmów kryptograficznych (np. bcrypt, Argon2) oraz z wykorzystaniem unikalnej "soli" (salt) dla każdego konta.
4. System musi posiadać mechanizmy chroniące publiczne punkty końcowe (np. logowanie, reset hasła) przed zautomatyzowanymi atakami typu Brute-Force, stosując czasową blokadę konta po określonej liczbie nieudanych prób logowania.
5. System musi zapewniać ochronę przed atakami typu injection (np. SQL Injection, XSS) poprzez walidację i sanityzację danych wejściowych zgodnie z wytycznymi OWASP.
6. Architektura API musi opierać się na zasadzie minimalnych uprawnień (PoLP). Każde żądanie do serwera (Endpoint) musi być niezależnie weryfikowane pod kątem posiadania przez użytkownika odpowiedniej roli lub tokenu.
7. System musi implementować bezpieczne zarządzanie sesją, w tym generowanie unikalnych tokenów (np. JWT), ich rotację oraz unieważnianie wszystkich aktywnych sesji natychmiast po zmianie hasła lub wylogowaniu.
8. System musi wdrażać restrykcyjną politykę CORS, zezwalając na zapytania do API wyłącznie z zaufanych i zdefiniowanych w konfiguracji domen (whitelisting).
9. System musi zapewniać bezpieczne przechowywanie danych uwierzytelniających oraz kluczy dostępowych (secrets) z wykorzystaniem dedykowanych mechanizmów zarządzania sekretami (np. vault), bez przechowywania ich w kodzie źródłowym.
10. System musi zapewniać automatyczne rejestrowanie logów bezpieczeństwa w niejawnym dzienniku (Audit Trail), obejmujących operacje takie jak logowanie, zmiany danych, zmiany ról oraz eksport danych, wraz z zapisem timestamp, adresu IP oraz identyfikatora użytkownika, przy jednoczesnym uniemożliwieniu ich modyfikacji z poziomu aplikacji.

3.2.3.Obszar Kompatybilności i Środowiska Klientckiego (autor: Yurii Morskyi)

1. Aplikacja nie może wymagać od użytkowników instalacji, wdrażania ani aktualizacji dodatkowego oprogramowania klienckiego na urządzeniach.
2. System musi zapewniać poprawne działanie na urządzeniach z systemami operacyjnymi Windows, macOS oraz Linux, pod warunkiem dostępności obsługiwanej przeglądarki internetowej.
3. System musi oficjalnie wspierać dwie najnowsze stabilne wersje główne (major) każdej z obsługiwanych przeglądarek. Wersje starsze mogą nie być wspierane, a system musi informować użytkownika o konieczności aktualizacji przeglądarki, jeśli wykryje wersję spoza zakresu wsparcia.
4. Interfejs użytkownika musi być responsywny i zapewniać pełną funkcjonalność na urządzeniach o rozdzielczości ekranu od 1280×720 pikseli wzwyż. Na

- urządzeniach o mniejszej rozdzielczości system musi pozostać czytelny i umożliwiać dostęp do kluczowych funkcji.
5. Złożone elementy interfejsu graficznego, takie jak roadmapy i wykresy, muszą renderować się płynnie w obsługiwanych przeglądarkach. Czas ładowania i pełnego wyrenderowania widoku z roadmapą zawierającą do 500 zadań nie może przekraczać 3 sekund.
 6. Interfejs użytkownika musi spełniać wymagania standardu WCAG 2.1 na poziomie AA w zakresie kontrastu kolorów, czytelności tekstu, etykietowania elementów interaktywnych oraz obsługi czytników ekranowych.
 7. Wszystkie kluczowe operacje użytkownika (nawigacja, tworzenie i edycja zadań, zmiana statusów, obsługa tablic) muszą być w pełni dostępne za pomocą klawiatury, bez konieczności użycia myszy.
 8. W przypadku, gdy przeglądarka użytkownika nie wspiera wymaganej funkcjonalności (np. WebSocket, Drag and Drop API), system musi wyświetlić czytelny komunikat informujący o ograniczeniu, zamiast generować błędy lub prezentować niepoprawny interfejs.
 9. Każdy widok, projekt, tablica oraz szczegóły zadania muszą posiadać unikalny, bezpośredni adres URL (deep link), umożliwiający udostępnianie linków między członkami zespołu oraz poprawne działanie przycisków nawigacji przeglądarki (wstecz/dalej).
 10. System musi obsługiwać kodowanie znaków UTF-8 we wszystkich polach tekstowych (tytuły, opisy, komentarze), zapewniając poprawne wyświetlanie i przechowywanie znaków alfabetów nielacińskich.

3.2.4. Obszar Skalowalności i Architektury (Scalability) **(autor: Bohdan Kliukovskyi)**

1. System musi opierać się na architekturze z wyraźnym podziałem warstw (Frontend, logika biznesowa API, Baza Danych), komunikujących się ze sobą wyłącznie poprzez udokumentowany interfejs.
2. System musi wykorzystywać technologię konteneryzacji (np. Docker) oraz orkiestracji (np. Kubernetes) w celu zapewnienia pełnej przenośności i standaryzacji środowisk wdrożeniowych.
3. System musi posiadać bezstanową warstwę logiki biznesowej (Stateless Backend), przechowując dane sesyjne w zewnętrznej, rozproszonej pamięci podręcznej (np. Redis), co umożliwi skalowanie poziome.
4. System musi wykorzystywać relacyjną bazę danych (RDBMS) wspierającą replikację danych w architekturze klastrowej, umożliwiając odseparowanie operacji zapisu od operacji odczytu (Read Replicas).
5. System musi przechowywać wszystkie pliki wgrywane przez użytkowników (załączniki, dokumenty) w zewnętrznej, skalowalnej usłudze przechowywania obiektów (Object Storage, np. S3), zamiast na dysku lokalnym serwera.
6. System musi implementować mechanizmy rozproszonego cachowania danych (np. Redis) dla wyników najbardziej obciążających zapytań, w celu redukcji czasu odpowiedzi i obciążenia bazy danych.
7. System musi wspierać dystrybucję zasobów statycznych (obrazy, skrypty, style) za pośrednictwem sieci dostarczania treści (CDN), aby zminimalizować opóźnienia sieciowe dla użytkowników końcowych.
8. System musi wykorzystywać asynchroniczne kolejki wiadomości (Message Queues, np. RabbitMQ, Kafka) do procesowania ciężkich zadań w tle (np. generowanie dużych raportów, wysyłka masowa powiadomień).

9. System musi stosować mechanizm paginacji oraz leniwego ładowania (lazy loading) dla wszystkich list danych (np. zadania na tablicy), aby zapobiec przeciążeniu interfejsu przy dużej ilości rekordów.
10. System musi udostępniać zintegrowane punkty kontrolne (Health Checks) oraz metryki wydajnościowe, umożliwiając systemom zewnętrznym monitorowanie stanu zdrowia każdego węzła architektury.

3.2.5. Obszar Wydajności i Integracji (Performance & Interoperability) (autor: Bohdan Krasovskyi)

1. System musi działać płynnie i bez opóźnień, zapewniając użytkownikowi natychmiastową reakcję na każde kliknięcie czy otwarcie widoku projektu.
2. Podczas przesuwania zadań (drag-and-drop) system musi wyraźnie pokazywać, gdzie można upuścić kartę, dbając o to, by zmiana statusu była zgodna z zasadami pracy w zespole.
3. Wszelkie połączenia z zewnętrznymi narzędziami analitycznymi muszą być stabilne, co gwarantuje, że aktualizacje systemu nie zepsują działających już raportów i zestawień.
4. Aplikacja musi być zoptymalizowana pod kątem pracy mobilnej, tak aby użytkownicy korzystający ze słabszego połączenia internetowego mogli sprawnie zarządzać zadaniami bez dużego zużycia danych.
5. System musi na bieżąco współpracować z narzędziami programistycznymi (np. GitHub), dzięki czemu postępy w tworzeniu kodu są od razu widoczne w System bez ręcznego wprowadzania danych.
6. Aktualizacja statusów zadań musi odbywać się automatycznie na podstawie sygnałów z zewnętrznych systemów technicznych, co pozwala zespołowi zaoszczędzić czas na manualne aktualizacje.
7. Powiadomienia e-mail muszą być dostarczane w sposób niezawodny i bezpieczny, gwarantując, że żadna ważna informacja o zmianach w zadaniu nie umknie uwadze użytkownika.
8. System musi umożliwiać łatwe przesyłanie informacji o projektach do komunikatorów takich jak Slack czy MS Teams, ułatwiając codzienną komunikację i współpracę w zespole.
9. System musi chronić serwer przed nadmiernym obciążeniem, blokując próby zbyt częstego wysyłania zapytań przez automatyczne skrypty, co zapewnia stały dostęp do aplikacji dla wszystkich pracowników.
10. Aplikacja musi automatycznie zwiększać swoją moc obliczeniową, gdy z systemu korzysta jednocześnie bardzo duża liczba osób, tak aby czas ładowania stron zawsze pozostawał na komfortowym poziomie.

3.3.Wymagania zgodności

3.3.1.Obszar Zgodności Prawnej i Ochrony Danych (autor: Oleksandr Barabash)

1. System musi przechowywać wrażliwe dane osobowe pracowników (numer PESEL, dane finansowe) w bazie danych w formie zaszyfrowanej, z wykorzystaniem silnych algorytmów kryptograficznych.
2. System musi umożliwiać Administratorowi (lub Użytkownikowi) wygenerowanie i pobranie wszystkich zgromadzonych danych osobowych danego użytkownika w ustrukturyzowanym formacie (plik CSV/JSON/PDF).
3. Podczas pierwszego logowania system musi wymuszać na użytkowniku akceptację polityki prywatności, a informacja o dacie i czasie akceptacji musi zostać zapisana w bazie danych.
4. System musi szyfrować wszystkie dane przesyłane pomiędzy aplikacją klienta a serwerem przy użyciu bezpiecznego protokołu komunikacyjnego, aby zapobiec przechwyceniu informacji.
5. System musi posiadać mechanizm trwałej anonimizacji danych osobowych Użytkownika (tzw. "prawo do bycia zapomnianym" zgodnie z RODO) na jego żądanie. Anonimizacja musi polegać na nieodwracalnym nadpisaniu danych identyfikujących (imię, nazwisko, e-mail) ciągiem losowych znaków, przy jednoczesnym zachowaniu spójności relacji w bazie danych dla celów statystycznych.
6. System musi umożliwiać Użytkownikowi wgląd w zaakceptowane regulaminy i polityki prywatności oraz dawać możliwość wycofania zgód marketingowych w dowolnym momencie z poziomu profilu użytkownika.
7. System musi automatycznie identyfikować i anonimizować lub usuwać dane osobowe użytkowników i klientów, z którymi nie odnotowano żadnej interakcji (logowania, transakcji) przez okres dłuższy niż 5 lat, zgodnie z polityką retencji danych.
8. W przypadku zmiany regulaminu lub polityki prywatności, system musi zablokować dostęp do głównych funkcji konta przy najbliższym logowaniu, aż do momentu zaakceptowania nowej wersji dokumentu przez Użytkownika.
9. System musi domyślnie blokować uruchomienie wszelkich opcjonalnych skryptów śledzących (cookies analityczne i marketingowe) do momentu udzielenia wyraźnej i aktywnej zgody przez Użytkownika poprzez dedykowany panel (Cookie Banner).
10. Interfejsy formularzy (rejestracji, edycji profilu) muszą wyraźnie oddzielać dane obowiązkowe (niezbędne do świadczenia usługi) od danych opcjonalnych. System nie może blokować utworzenia konta w przypadku niepodania danych opcjonalnych.

3.3.2.Obszar Zgodności z Politykami Bezpieczeństwa Korporacyjnego (autor: Stanislav Smaha)

1. System musi być chroniony przez zaporę aplikacyjną (WAF – Web Application Firewall) oraz mechanizmy mitygacji ataków DDoS na poziomie infrastruktury sieciowej (korporacyjnej lub chmurowej).
2. Architektura systemu musi zapewniać izolację sieciową poprzez wydzielenie prywatnych podsieci (np. VPC), w których baza danych oraz serwery backendowe nie posiadają bezpośredniego dostępu z publicznej sieci Internet.

3. Dane przechowywane w bazach danych oraz magazynach plików muszą być szyfrowane w spoczynku (Data-at-Rest) z wykorzystaniem algorytmu AES-256 oraz zarządzania kluczami poprzez dedykowane usługi (np. KMS, Key Vault).
4. Komunikacja pomiędzy komponentami systemu w sieci prywatnej musi być szyfrowana z wykorzystaniem mechanizmu mTLS (Mutual TLS), zgodnie z założeniami architektury Zero Trust.
5. System musi umożliwiać eksport logów bezpieczeństwa i audytowych w ujednoliconym formacie do systemów klasy SIEM (np. Splunk, IBM QRadar) w czasie rzeczywistym.
6. Powiadomienia e-mail generowane przez system muszą być wysyłane z wykorzystaniem protokołu SMTP z wymuszonym szyfrowaniem TLS w wersji co najmniej 1.2 (np. STARTTLS).
7. Proces wytwórczy oprogramowania musi obejmować automatyczne skanowanie kodu źródłowego oraz testy dynamiczne przed każdym wdrożeniem na środowisko produkcyjne, w celu wykrywania podatności bezpieczeństwa.
8. System musi być regularnie analizowany pod kątem znanych podatności (CVE) w komponentach open-source (Software Composition Analysis), z automatycznym blokowaniem wdrożeń zawierających podatności o poziomie Critical lub High.
9. Wymiana danych poprzez integracje zewnętrzne musi być realizowana za pośrednictwem bramy API (API Gateway), zapewniającej autoryzację opartą na tokenach lub certyfikatach oraz walidację struktury danych (schema validation).
10. System musi umożliwiać przeprowadzanie cyklicznych testów penetracyjnych przez niezależne zespoły, bez wpływu na dostępność środowiska produkcyjnego.

3.3.3. Obszar Zgodności ze Standardami Webowymi i Przeglądarkami (autor: Yurii Morskyi)

1. Kod warstwy klienckiej (Frontend) musi być zgodny ze specyfikacją WHATWG HTML Living Standard. Znaczniki HTML muszą przechodzić walidację W3C Markup Validation Service bez błędów krytycznych.
2. Logika aplikacji po stronie przeglądarki musi być napisana zgodnie ze standardem ECMAScript 2020 (ES11) lub nowszym. Kod nie może wykorzystywać niestandardowych rozszerzeń specyficznych dla pojedynczego silnika przeglądarki.
3. Stylowanie interfejsu musi być zgodne ze specyfikacjami modułów CSS3 publikowanymi przez W3C. Arkusze stylów nie mogą zawierać prefiksów vendor (np. -webkit-, -moz-) jako jedynej implementacji danej właściwości — każdy prefiks musi być uzupełniony o wersję standardową.
4. Elementy interaktywne interfejsu muszą być oznakowane zgodnie ze specyfikacją WAI-ARIA 1.2, zapewniając poprawną semantykę ról, stanów i właściwości dla technologii asystujących.
5. Aplikacja musi przechodzić testy zgodności i poprawności renderowania w przeglądarkach: Google Chrome, Mozilla Firefox, Apple Safari oraz Microsoft Edge, w zakresie dwóch najnowszych stabilnych wersji głównych każdej z nich.
6. Kod aplikacji nie może wykorzystywać interfejsów API przeglądarki oznaczonych jako przestarzałe (deprecated) w dokumentacji MDN Web Docs. W przypadku konieczności użycia API o ograniczonym wsparciu, wymagane jest zastosowanie mechanizmu feature detection z odpowiednim fallbackiem.

7. Aplikacja musi spełniać następujące progi metryk Core Web Vitals mierzonych narzędziem Lighthouse: Largest Contentful Paint (LCP) $\leq 2,5$ s, Cumulative Layout Shift (CLS) $\leq 0,1$, Interaction to Next Paint (INP) ≤ 200 ms.
8. Struktura dokumentu HTML musi wykorzystywać znaczniki semantyczne (np. `<nav>`, `<main>`, `<article>`, `<section>`) zamiast generycznych kontenerów `<div>`, wszędzie tam, gdzie istnieje odpowiedni znacznik semantyczny.
9. Wszystkie zasoby statyczne aplikacji (skrypty JavaScript, arkusze CSS, obrazy) muszą być dostarczane z zastosowaniem kompresji (gzip lub Brotli) oraz mechanizmów cache'owania HTTP (nagłówki Cache-Control, ETag).
10. Aplikacja musi obsługiwać Content Security Policy (CSP) na poziomie nagłówków HTTP, blokując wykonywanie skryptów inline oraz ładowanie zasobów z niezaufanych źródeł zewnętrznych.

3.3.4. Obszar Zgodności Architektonicznej i Interfejsów API **(autor: Bohdan Kliukovskyi)**

1. System musi udostępniać główne interfejsy programistyczne w architekturze RESTful, wykorzystując standardowe metody HTTP (GET, POST, PUT, PATCH, DELETE) do operacji na zasobach.
2. System musi wykorzystywać format JSON jako jedyny standard wymiany danych (zarówno dla żądań, jak i odpowiedzi) w komunikacji poprzez REST API.
3. System musi implementować ścisły mechanizm wersjonowania usług (np. poprzez prefiks w URI `/v1/`), zapewniając pełną kompatybilność wsteczną dla istniejących klientów i integracji.
4. System musi zapewniać interaktywną i automatycznie generowaną dokumentację API zgodną ze specyfikacją OpenAPI 3.0 (np. Swagger UI), dostępną dla programistów w czasie rzeczywistym.
5. System musi implementować ustandaryzowany format zwracania błędów zgodny z RFC 7807, zawierający unikalny kod błędu, opis oraz szczegóły walidacji w obiekcie JSON.
6. System musi stosować spójny mechanizm paginacji dla wszystkich punktów końcowych zwracających kolekcje danych, dołączając do odpowiedzi metadane o liczbie rekordów.
7. System musi informować systemy zewnętrzne o limitach zapytań poprzez standardowe nagłówki HTTP (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset).
8. System musi kryptograficznie podpisywać każdy ładunek wychodzący (Webhooks) przy użyciu algorytmu HMAC, umożliwiając odbiorcom weryfikację autentyczności nadawcy.
9. System musi wspierać mechanizm kluczy idempotencji (Idempotency-Key) w nagłówkach żądań dla operacji zmieniających stan, zapobiegając powtórnemu przetworzeniu tych samych danych.
10. System musi nakładać sztywne limity wielkości przesyłanego ładunku (Payload Size Limits) dla zapytań JSON oraz plików (Multipart), w celu ochrony zasobów pamięci serwera.

3.3.5.Obszar Zgodności z Metodykami Zarządzania Projektami (autor: Bohdan Krasovski)

1. System musi wspierać elastyczne zarządzanie projektami, pozwalając zespołowi na szybką adaptację do zmian i sprawne planowanie kolejnych etapów prac bez zbędnych formalności.
2. System musi umożliwiać gromadzenie i porządkowanie wszystkich potrzeb projektowych w jednym miejscu, co ułatwia zespołowi wybór najważniejszych zadań do realizacji w pierwszej kolejności.
3. Użytkownicy muszą mieć możliwość planowania pracy w krótkich cyklach, dzięki czemu zespół może regularnie dostarczać nowe, gotowe do użycia funkcje i szybko zbierać opinie.
4. System musi zapewniać czytelną wizualizację postępów na wspólnych tablicach, aby każdy od razu widział, co zostało już zrobione, a co wymaga uwagi w danym momencie.
5. System musi umożliwiać zespołowi wspólne ocenianie stopnia skomplikowania zadań, co pozwala na lepsze dopasowanie ilości pracy do realnych możliwości pracowników.
6. System musi umożliwiać przejrzyste układanie planu pracy poprzez wygodne dzielenie dużych celów na konkretne, mniejsze kroki oraz zadania zrozumiałe dla całego zespołu.
7. System powinien automatycznie podpowiadać, ile pracy zespół może realnie wykonać, bazując na efektach z poprzednich tygodni, co znacznie ułatwia dotrzymywanie terminów.
8. Sposób organizacji pracy w systemie musi gwarantować, że każdy zakończony etap prac kończy się powstaniem działającej i przetestowanej części oprogramowania.
9. Kierownicy projektów muszą mieć wyłączne prawo do zarządzania harmonogramem i oficjalnego zatwierdzania końcowych efektów prac, aby dbać o porządek i spójność w projekcie.
10. Każdy pracownik musi mieć prosty sposób na pokazywanie postępów swojej pracy na tablicy, dzięki czemu cały zespół jest zawsze na bieżąco bez konieczności zwoływania dodatkowych spotkań.